

Practical Assignment-3

NAME Jeet Pandya

DIV 2

STUDENT ID 24BCA095

Q1 Create and use a package mypackage with a Greeting class that displays Hello World.

PROGRAM

mypackage/Greeting.java

```
package mypackage;

public class Greeting {
    public void display() {
        System.out.println("Hello World.");
    }
}
```

Q1_UseGreeting.java

```
import mypackage.Greeting;

public class Q1_UseGreeting {
    public static void main(String[] args) {
        Greeting greeting = new Greeting();
        greeting.display();
    }
}
```

OUTPUT

Hello World.

Q2 Create packages pkgA and pkgB and demonstrate public, protected, and default access modifiers.

PROGRAM

pkgA/AccessDemo.java

```
package pkgA;

public class AccessDemo {
    public void publicMethod() {
        System.out.println("Public method is accessible.");
    }

    protected void protectedMethod() {
        System.out.println("Protected method is accessible to a subclass.");
    }

    void defaultMethod() {
        System.out.println("Default method is accessible only inside pkgA.");
    }
}
```

pkgB/AccessTest.java

```
package pkgB;

import pkgA.AccessDemo;

public class AccessTest extends AccessDemo {
    public void testAccess() {
        publicMethod();
        protectedMethod();
    }
}
```

```
// defaultMethod(); does not compile because it belongs to package pkgA.
}

public static void main(String[] args) {
    new AccessTest().testAccess();
    System.out.println("Default method cannot be accessed from pkgB.");
}
}
```

OUTPUT

```
Public method is accessible.
Protected method is accessible to a subclass.
Default method cannot be accessed from pkgB.
```

Q3 Create shapes.Circle and graphics.Circle and print both areas using fully qualified names.

PROGRAM

shapes/Circle.java

```
package shapes;

public class Circle {
    private double radius = 5;

    public double area() {
        return Math.PI * radius * radius;
    }
}
```

graphics/Circle.java

```
package graphics;

public class Circle {
    private double radius = 7;

    public double area() {
        return Math.PI * radius * radius;
    }
}
```

Q3_CircleAreas.java

```
public class Q3_CircleAreas {
    public static void main(String[] args) {
        shapes.Circle shapeCircle = new shapes.Circle();
        graphics.Circle graphicCircle = new graphics.Circle();

        System.out.println("shapes.Circle area : " + shapeCircle.area());
        System.out.println("graphics.Circle area : " + graphicCircle.area());
    }
}
```

OUTPUT

```
shapes.Circle area : 78.53981633974483
graphics.Circle area : 153.93804002589985
```

Q4 Demonstrate String, Math, and System classes from java.lang.

PROGRAM

Q4_JavaLang.java

```
public class Q4_JavaLang {
    public static void main(String[] args) {
        String text = "Hello Java";
        System.out.println("Length : " + text.length());
        System.out.println("Square root : " + Math.sqrt(81));
        System.out.println("Power : " + Math.pow(2, 5));
    }
}
```

```
        System.out.println("Output printed using System.");
    }
}
```

OUTPUT

```
Length : 10
Square root : 9.0
Power : 32.0
Output printed using System.
```

Q5 Convert int and double values to wrapper objects and back using boxing and unboxing.

PROGRAM

Q5_WrapperClasses.java

```
public class Q5_WrapperClasses {
    public static void main(String[] args) {
        int number = 25;
        double price = 99.5;

        Integer boxedNumber = number;
        Double boxedPrice = price;
        int unboxedNumber = boxedNumber;
        double unboxedPrice = boxedPrice;

        System.out.println("Integer object : " + boxedNumber);
        System.out.println("Double object : " + boxedPrice);
        System.out.println("Sum : " + (unboxedNumber + boxedPrice));
        System.out.println("Price after tax : " + (unboxedPrice * 1.18));
    }
}
```

OUTPUT

```
Integer object : 25
Double object : 99.5
Sum : 124.5
Price after tax : 117.41
```

Q6 Create a thread by extending Thread and print Running Thread five times.

PROGRAM

Q6_ExtendsThread.java

```
class RunningThread extends Thread {
    public void run() {
        for (int count = 1; count <= 5; count++) {
            System.out.println("Running Thread");
        }
    }
}

public class Q6_ExtendsThread {
    public static void main(String[] args) throws InterruptedException {
        Thread thread = new RunningThread();
        thread.start();
        thread.join();
    }
}
```

OUTPUT

```
Running Thread
Running Thread
Running Thread
Running Thread
Running Thread
```

Q7 Implement Runnable to print numbers 1 to 10 while the main thread prints alphabets A to J.

PROGRAM

Q7_RunnableThreads.java

```
class NumberTask implements Runnable {
    public void run() {
        for (int number = 1; number <= 10; number++) {
            System.out.println("Number : " + number);
        }
    }
}

public class Q7_RunnableThreads {
    public static void main(String[] args) throws InterruptedException {
        Thread numberThread = new Thread(new NumberTask());
        numberThread.start();

        for (char letter = 'A'; letter <= 'J'; letter++) {
            System.out.println("Alphabet : " + letter);
        }
        numberThread.join();
    }
}
```

OUTPUT

```
Number : 1
Number : 2
Number : 3
Number : 4
Number : 5
Number : 6
Number : 7
Number : 8
Number : 9
Number : 10
Alphabet : A
Alphabet : B
Alphabet : C
Alphabet : D
Alphabet : E
Alphabet : F
Alphabet : G
Alphabet : H
Alphabet : I
Alphabet : J
```

[Note: Thread output order may vary.]

Q8 Create threads with MIN_PRIORITY, NORM_PRIORITY, and MAX_PRIORITY and observe execution order.

PROGRAM

Q8_ThreadPriorities.java

```
class PriorityTask extends Thread {
    PriorityTask(String name, int priority) {
        super(name);
        setPriority(priority);
    }

    public void run() {
        for (int count = 1; count <= 3; count++) {
            System.out.println(getName() + " priority " + getPriority());
        }
    }
}
```

```

public class Q8_ThreadPriorities {
    public static void main(String[] args) throws InterruptedException {
        Thread low = new PriorityTask("Low", Thread.MIN_PRIORITY);
        Thread normal = new PriorityTask("Normal", Thread.NORM_PRIORITY);
        Thread high = new PriorityTask("High", Thread.MAX_PRIORITY);

        low.start();
        normal.start();
        high.start();
        low.join();
        normal.join();
        high.join();
        System.out.println("Priority is only a scheduling hint; order may vary.");
    }
}

```

OUTPUT

```

High priority 10
High priority 10
High priority 10
Normal priority 5
Normal priority 5
Normal priority 5
Low priority 1
Low priority 1
Low priority 1
Priority is only a scheduling hint; order may vary.
[Note: Thread priority is a scheduling hint; order may vary.]

```

Q9 Create two threads to print even and odd numbers up to 20.

PROGRAM

Q9_EvenOddThreads.java

```

class EvenTask extends Thread {
    public void run() {
        for (int number = 2; number <= 20; number += 2) {
            System.out.println("Even : " + number);
        }
    }
}

class OddTask extends Thread {
    public void run() {
        for (int number = 1; number <= 19; number += 2) {
            System.out.println("Odd : " + number);
        }
    }
}

public class Q9_EvenOddThreads {
    public static void main(String[] args) throws InterruptedException {
        Thread even = new EvenTask();
        Thread odd = new OddTask();
        even.start();
        odd.start();
        even.join();
        odd.join();
    }
}

```

OUTPUT

```

Odd : 1
Odd : 3
Odd : 5
Odd : 7
Odd : 9

```

```

Odd : 11
Odd : 13
Odd : 15
Odd : 17
Odd : 19
Even : 2
Even : 4
Even : 6
Even : 8
Even : 10
Even : 12
Even : 14
Even : 16
Even : 18
Even : 20

```

[Note: Even and odd output order may vary.]

Q10 Create producer and consumer threads using an ArrayList and simple thread coordination.

PROGRAM

Q10_ProducerConsumer.java

```

import java.util.ArrayList;
import java.util.Random;

public class Q10_ProducerConsumer {
    public static void main(String[] args) throws InterruptedException {
        ArrayList<Integer> numbers = new ArrayList<>();
        Random random = new Random();

        Thread producer = new Thread(() -> {
            for (int count = 1; count <= 5; count++) {
                numbers.add(random.nextInt(100));
            }
        });
        Thread consumer = new Thread(() -> {
            for (int number : numbers) {
                System.out.println("Consumed : " + number);
            }
        });

        producer.start();
        producer.join();
        consumer.start();
        consumer.join();
    }
}

```

OUTPUT

```

Consumed : 85
Consumed : 40
Consumed : 86
Consumed : 82
Consumed : 54

```

Q11 Create an ArrayList of names, add and remove names, and display the result.

PROGRAM

Q11_ArrayList.java

```

import java.util.ArrayList;

public class Q11_ArrayList {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Aman");
    }
}

```

```

        names.add("Bhavna");
        names.add("Chirag");
        names.remove("Bhavna");

        for (String name : names) {
            System.out.println(name);
        }
    }
}

```

OUTPUT

```

Aman
Chirag

```

Q12 Store integers in a LinkedList and perform insertion and deletion.

PROGRAM

```

Q12_LinkedList.java
import java.util.LinkedList;

public class Q12_LinkedList {
    public static void main(String[] args) {
        LinkedList<Integer> numbers = new LinkedList<>();
        numbers.add(10);
        numbers.add(30);
        numbers.addFirst(5);
        numbers.add(1, 20);
        numbers.remove(Integer.valueOf(30));
        System.out.println(numbers);
    }
}

```

OUTPUT

```

[5, 20, 10]

```

Q13 Create a Vector of book titles and display it using Enumeration.

PROGRAM

```

Q13_VectorEnumeration.java
import java.util.Enumeration;
import java.util.Vector;

public class Q13_VectorEnumeration {
    public static void main(String[] args) {
        Vector<String> books = new Vector<>();
        books.add("Java Basics");
        books.add("Data Structures");
        books.add("Operating Systems");

        Enumeration<String> titles = books.elements();
        while (titles.hasMoreElements()) {
            System.out.println(titles.nextElement());
        }
    }
}

```

OUTPUT

```

Java Basics
Data Structures
Operating Systems

```

Q14 Create a HashSet of unique city names and try adding duplicates.

PROGRAM

Q14_HashSet.java

```
import java.util.HashSet;

public class Q14_HashSet {
    public static void main(String[] args) {
        HashSet<String> cities = new HashSet<>();
        cities.add("Delhi");
        cities.add("Mumbai");
        cities.add("Delhi");
        cities.add("Pune");
        System.out.println(cities);
    }
}
```

OUTPUT

```
[Delhi, Pune, Mumbai]
```

Q15 Create a TreeSet of integers and print them in sorted order.

PROGRAM

Q15_TreeSet.java

```
import java.util.TreeSet;

public class Q15_TreeSet {
    public static void main(String[] args) {
        TreeSet<Integer> numbers = new TreeSet<>();
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        System.out.println(numbers);
    }
}
```

OUTPUT

```
[10, 20, 30, 40]
```

Q16 Add numbered tasks to a PriorityQueue and print them in ascending order.

PROGRAM

Q16_PriorityQueue.java

```
import java.util.PriorityQueue;

public class Q16_PriorityQueue {
    public static void main(String[] args) {
        PriorityQueue<Integer> tasks = new PriorityQueue<>();
        tasks.add(40);
        tasks.add(10);
        tasks.add(30);
        tasks.add(20);

        while (!tasks.isEmpty()) {
            System.out.println(tasks.poll());
        }
    }
}
```

OUTPUT

```
10
20
30
40
```

Q17 Use Iterator and ListIterator to traverse an ArrayList forward and backward.

PROGRAM

Q17_IteratorListIterator.java

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.ListIterator;

public class Q17_IteratorListIterator {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        Iterator<Integer> iterator = numbers.iterator();
        while (iterator.hasNext()) {
            System.out.println("Iterator : " + iterator.next());
        }

        ListIterator<Integer> listIterator = numbers.listIterator();
        while (listIterator.hasNext()) {
            System.out.println("Forward : " + listIterator.next());
        }
        while (listIterator.hasPrevious()) {
            System.out.println("Backward : " + listIterator.previous());
        }
    }
}
```

OUTPUT

```
Iterator : 10
Iterator : 20
Iterator : 30
Forward : 10
Forward : 20
Forward : 30
Backward : 30
Backward : 20
Backward : 10
```

Q18 Store student roll numbers and names in a HashMap and print all entries.

PROGRAM

Q18_HashMap.java

```
import java.util.HashMap;

public class Q18_HashMap {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();
        students.put(101, "Aman");
        students.put(102, "Bhavna");
        students.put(103, "Chirag");

        for (var entry : students.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

OUTPUT

```
101 : Aman
102 : Bhavna
103 : Chirag
```

Q19 Store countries and capitals in a LinkedHashMap and show insertion order.

PROGRAM

Q19_LinkedHashMap.java

```
import java.util.LinkedHashMap;

public class Q19_LinkedHashMap {
    public static void main(String[] args) {
        LinkedHashMap<String, String> countries = new LinkedHashMap<>();
        countries.put("India", "New Delhi");
        countries.put("Japan", "Tokyo");
        countries.put("France", "Paris");

        for (var entry : countries.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

OUTPUT

```
India : New Delhi
Japan : Tokyo
France : Paris
```

Q20 Store product names and prices in a TreeMap and print them sorted by product name.

PROGRAM

Q20_TreeMap.java

```
import java.util.TreeMap;

public class Q20_TreeMap {
    public static void main(String[] args) {
        TreeMap<String, Double> products = new TreeMap<>();
        products.put("Keyboard", 1200.0);
        products.put("Monitor", 8500.0);
        products.put("Mouse", 600.0);

        for (var entry : products.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

OUTPUT

```
Keyboard : 1200.0
Monitor : 8500.0
Mouse : 600.0
```